



POLITECNICO DI TORINO

MSC MECHATRONICS ENGINEERING

ROBOTICS

INTELLIGENT CONTROL OF ROBOTS WITH MINIMAL POWER
CONSUMPTION IN PICK-AND-PLACE OPERATIONS

GROUP:

CESARE MESSA, S361616

GOFFREDO PIAZZA, S360031

GABRIELE LISCI, S362710

PIETRO FILOMENO, S360042

A.Y 2025-2026

Introduction

Energy efficiency is a relevant aspect in industrial automation, especially for repetitive pick-and-place operations used in assembly such as, packaging, manufacturing and quality control. In these tasks, the robot configuration strongly influences the required joint torques and, consequently, the overall energy consumption. The work proposed in the analyzed paper focuses on reducing the power consumption of robotic manipulators by combining kinematic analysis, inverse statics, and reinforcement learning techniques. In this project, these concepts are applied to the industrial robotic arm ABB IRB 1600. The focus of the project is to evaluate different configurations selected based on estimated torque. The best configuration is then fed into a Reinforcement Learning model, so that the robot can learn how to reproduce and optimize the desired motion.

Robot Model

The first part of the project required defining the manipulator's kinematic model based on the reference paper. However, a critical issue emerged: the provided Denavit-Hartenberg (DH) parameters were geometrically incompatible with the standard CAD meshes of the real ABB IRB 1600 robot. Applying the paper's raw parameters resulted in a distorted kinematic chain and incorrect center-of-mass (CoM) positioning, which would have compromised further project phases.

To keep a neat and computationally light code, instead of implementing complex offset matrices in MATLAB or roto-translational blocks in Simulink, a CAD pre-processing strategy was adopted. High-fidelity STL meshes were imported into Autodesk Fusion, re-centered, and aligned with their respective local joint reference frames. This reverse-engineering process extracted the actual operational lengths for the a and d DH parameters (e.g., $a_{12} = 0.1515$ m, $d_{45} = 0.3548$ m), completely redefining the structural table while maintaining the required base topology.

Following the CAD optimization, the refined kinematic chain was implemented in MATLAB. Through custom scripts (`creairb.m`), the `rigidBodyTree` object was assembled. Each link was assigned precise inertial properties (mass, CoM, inertia) and bounded to the generated `rigidBodyJoint` elements. For realistic visualization, the pre-processed STL files were attached using the `addVisual` function.

To move to multibody dynamics, the `smimport` function was used to automatically create a Simscape Multibody model. Since the model generated by the function was not well organized, the Simulink environment was then cleaned up and reorganized. In particular, the transform blocks were simplified, the joint sensing and actuation ports were correctly connected, and the layout was divided into clearer modules. This created a solid dynamic base for the following Forward and Inverse Kinematics phases.

Forward Kinematics And Inverse Kinematics

The Inverse Kinematics (IK) problem consists of finding the joint configurations that allow the end effector to reach a desired target pose. In the reference paper, the case study is simplified by considering a planar motion of the end effector in the XZ plane. This is achieved by allowing only three joints to vary, namely J2, J3, and J5, while the remaining joints, J1, J4, and J6, are kept fixed at zero joint angle. This reduces power consumption because those joints are no longer required to counteract gravitational torque. As a result, their contribution to the gravity torque vector for a given configuration becomes zero.

The implementation developed in this project maintains this feature. However, an additional constraint on the orientation of the end effector is introduced. This makes the case study more

specific, since the robot is required not only to reach the target position, but also to keep the end effector in a suitable orientation (z-axis of the tool pointing downward) for vertical operations.

Under these assumptions, the IK problem is redundant. Indeed, the reduced joint space is described by three joint variables, namely q_2 , q_3 , and q_5 , while the main task is a target position expressed by only two independent coordinates, X and Z . The orientation is not imposed as a complete equality constraint on roll, pitch, and yaw, but only with an inequality constraint the angle between z-axis of the tool and z-axis of base frame. So the number of joint variables is bigger than the number of independent task necessary to express variables.

The aim of the algorithm is to explore a finite number of joint configuration in order to identify the admissible final configurations that allow the robot to reach the target position starting from the same home configuration, namely $q = (0, 0, 0, 0, 0, 0)$.

The total joint space is represented by the range given by joints limits from the rigid body tree. The algorithm discretizes the joints space in order to explore a limited set of solutions inside the working space. For each joint range discretization step is equal to $\frac{\text{jointlimitsrange}}{N}$. Then, for loops set a value for each movable joint, and the Forward Kinematics is computed for every joint combination. Since three joints are varied, the total number of tested configurations is N^3 . The end effector pose is computed through Transformation matrices obtained by the Rigid Body Tree. The pose, taken in consideration at every recursion, is compared with the desired target pose, checking both position and orientation.

Since an orientation constraint was added, discretization steps were increased from $N=50$ (paper) to $N=100$, in order to get more admissible configurations. Those final configurations are stored and later evaluated through an Inverse Statics algorithm to estimate power consumption.

Inverse Statics

The inverse statics algorithm provides the total torque required to maintain the manipulator in equilibrium for a specific joint configuration. Neglecting inertial and dynamic effects under static assumptions, the gravitational torque τ_g is computed with the MATLAB function **gravityTorque** applied to the robot's rigid body tree model. To account for joint elasticity, a simplified stiffness torque is introduced as a linear function of the angular displacement, $\tau_{s,j}(q) = k_j \cdot q_j$. The coefficients k_j were selected empirically due to their absence in the reference literature and official documentation, serving as an approximation to evaluate stiffness influence. For each configuration, the total torque magnitude per joint is defined as $\tau_{\text{tot},j} = |\tau_{g,j}| + |\tau_{s,j}|$. To identify the most efficient configuration, a scalar performance index is defined by aggregating contributions across all six joints, $J(q) = \sum_{j=1}^6 \tau_{\text{tot},j}$, leading to the optimization problem $q_{\text{opt}} = \arg \min_q J(q)$. Results demonstrate that the first configuration minimizes this objective function, proving to be the most advantageous from an inverse statics perspective.

RL Simulation

Reinforcement Learning (RL) is employed to train an agent to make autonomous, optimal decisions by maximizing a reward signal over time. A dedicated Simulink model was developed to serve as the training environment. This model is composed by four primary functional blocks: the Robot Model, the Reward Subsystem, the RL Agent block, and the Joint Drive Function block.

The Joint Drive Function block acts as the actuation interface between the RL agent and the mechanical structure. It processes the control actions issued by the agent, computing the corresponding joint torques applied to the multibody system and monitoring the instantaneous electrical and mechanical power absorption.

An RL Twin Delayed (TD3) agent was generated via MATLAB. Its action space is a 6-dimensional continuous vector (6×1) bounded within $[-100, 100]$, which represents the manipulator’s joint torques, while the observation space is a 32-dimensional vector (32×1) describing the environment state. To ensure numerical stability during backpropagation, all observation signals are normalized within $[-1, 1]$ via scaling gains. The TD3 architecture features an Actor, two distinct Critics, and a shared input layer for optimized feature extraction. Hyperparameters were iteratively tuned over sequential training sessions, optimizing the Target Smooth Factor, the Target Policy Smooth Model variance and its decay rate, the Actor/Critic learning rates, and the exploration noise standard deviation along with its associated decay rate.

A new reward function was designed to improve the baseline from the reference paper, which only penalized the Euclidean distance and the power consumption. The Cartesian distance was kept as the main priority, while the power penalty was replaced by the joint velocity norm. Since power is proportional to the square of velocity ($P \propto \omega^2$), penalizing the velocity vector directly attenuates kinetic peaks. Additionally, a joint configuration penalty (second highest priority) was introduced via the forward kinematics detailed in Section X. This resolves geometric ambiguities, as a specific joint configuration gives a unique end-effector pose, while inverse mapping can have multiple solutions. To shape the agent’s behavior, specific auxiliary bonuses were implemented ($\Delta x = x_{\text{target}} - x_{\text{current}}$).

$$R = -15 \cdot \|\Delta p\|^2 - 0.1 \cdot \|\Delta q\|^2 - 5 \times 10^{-4} \cdot \|\dot{q}\|^2 + e^{(-50 \cdot \|\Delta p\|^2)}$$

$$\begin{array}{ll} \text{Bonus}_{q(5)} = 0.3 & 0 \leq q(5) \leq 1.7 \\ \text{Bonus}_{q(1)} = 0.2 & -0.05 \leq q(1) \leq 0.05 \\ \text{Bonus}_{\Delta p} = 1.5 \cdot e^{(1 \times 10^{-3} \cdot \|\dot{q}\|^2)} & \Delta p \leq 0.1 \end{array}$$

Training Results

Finally, the training simulation was executed. While reference studies simulated over 200 episodes at 1–1.5 seconds each, this setup utilizes 200 episodes with an extended duration of 2 seconds. This longer episode duration helps the agent better understand the task and accelerates convergence. Although the agent demonstrated a clear learning trend, the final performance revealed minor joint oscillations and a residual error relative to the target configuration, primarily due to the capped 200 episode training budget. Nonetheless, the results obtained within this constrained framework remain highly satisfactory.

Conclusion and Future Work

This project investigated the optimization of an industrial robotic manipulator for energy-efficient pick-and-place operations, integrating inverse kinematics, inverse statics, and reinforcement learning within a Simulink-based TD3 environment. The introduction of a downward end-effector orientation constraint, the redefinition of the DH parameters, and the recomputation of the reward function successfully yielded reasonable results.

As future work, a second training session is suggested to resolve the residual issues resulting from the first session. Moreover, since the model is designed to support a complete multi-stage pick-and-place task, further training sessions focusing on the overall pick and place operation can be directly

AI was used in the development of this work for code debugging, support in explaining the report concepts, and production of the presentation video. The repository is available at the following link: <https://github.com/piazzagoffredo-ai/ABB-IRB-1600-Robotics-Project.git>